Name: Gunjan Chandnani Roll No : 13 Div : D20A

EXPERIMENT NO: 04

AIM: Hands on Solidity Programming Assignments for creating Smart Contract**s**

1. Primitive Data Types, Variables, Functions (pure, view)

In Solidity, primitive data types are the basic building blocks used to store simple values in smart contracts.

Common Primitive Types:

- **uint / int** – Unsigned and signed integers of different sizes such as `uint256`, `int128`.
- **bool** – Stores logical values: `true` or `false`.
- **address** – Holds a 20-byte Ethereum wallet or contract address.
- **bytes / string** – Used to store binary data or text.

Types of Variables:

- **State Variables** – Stored permanently on the blockchain. Defined outside functions.
- **Local Variables** – Temporary variables created inside functions.
- **Global Variables** – Special built-in variables like `msg.sender`, `msg.value`, `block.timestamp`.

Function Types:

- **pure** – Cannot read or modify blockchain data. Works only with inputs and calculations.
- **view** – Can read blockchain data but cannot modify it.

These keywords help reduce gas usage and maintain security.

2. Inputs and Outputs to Functions

Solidity functions can take **inputs (parameters)** and return **outputs (return values)**.

- **Inputs** allow users or other contracts to send data to the function.
- **Outputs** allow functions to send results back after processing.
- Solidity also supports **named return variables**, which improve readability and debugging.

Example uses include checking balances, validating transfers, or returning calculation results.

3. Visibility, Modifiers, and Constructors

Function Visibility

Defines who can access a function:

- **public** – Accessible from inside and outside the contract.
- **private** – Accessible only within the same contract.
- **internal** – Accessible within the contract and derived contracts.
- **external** – Callable only from outside the contract.

Modifiers

Modifiers are reusable logic blocks used to change or restrict function behavior.
Common use:

- **onlyOwner** – Restricts access to contract owner.

Constructors

A constructor is a special function that runs **only once** when the contract is deployed. It is mainly used for:

- Setting the contract owner
- Initializing important values

4. Control Flow – if-else and Loops

Control flow statements manage the execution order of code.

- **if-else** – Used for decision making, such as verifying balances before transfers.
- **Loops (for, while, do-while)** – Used for repeated execution of code.

⚠ **Important:** Excessive loops increase gas consumption and can make transactions expensive.

5. Data Structures – Arrays, Mappings, Structs, Enums

Arrays

Store ordered collections of elements.

- Can be **fixed** or **dynamic** in size.
- Example: List of registered user addresses.

Mappings

Key-value storage structure for fast lookup.

- Example: `mapping(address => uint)` to store balances.
- Cannot be iterated directly.

Structs

Custom data types that group multiple variables.

- Example:

```
struct Player {
   string name;
   uint score;
}
```

Enums

Define a set of constant values to improve readability.

- Example: `enum Status { Pending, Active, Closed }`
-

6. Data Locations

Solidity provides three main data locations that affect gas cost and performance:

- **storage** – Permanent data stored on the blockchain (state variables).
- **memory** – Temporary data stored only during function execution.
- **calldata** – Temporary, read-only data used mainly for external function parameters. It is the most gas-efficient.

Choosing the correct data location is important for optimization.

7. Transactions – Ether, Wei, Gas, and Sending Transactions

Ether and Wei

- **Ether** is the main cryptocurrency of Ethereum.
- **Wei** is the smallest unit of Ether.
- `1 Ether = 10^18 Wei`

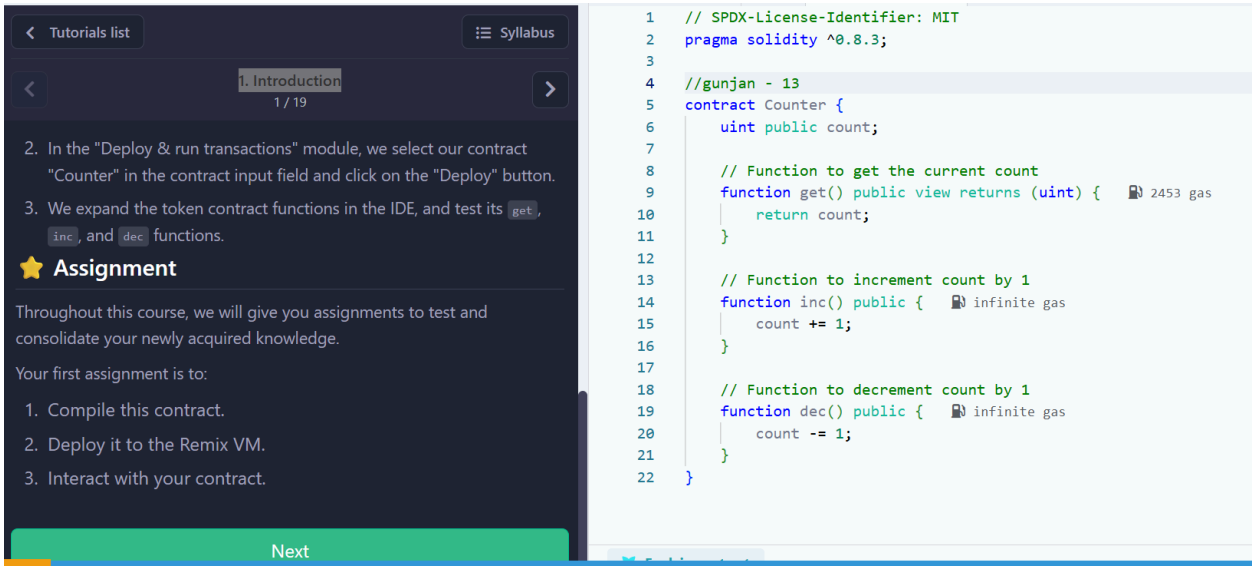
Gas and Gas Price

- **Gas** measures computational work required to execute transactions.
- **Gas Price** determines how much Ether is paid per gas unit.
- Higher gas price → Faster transaction priority.

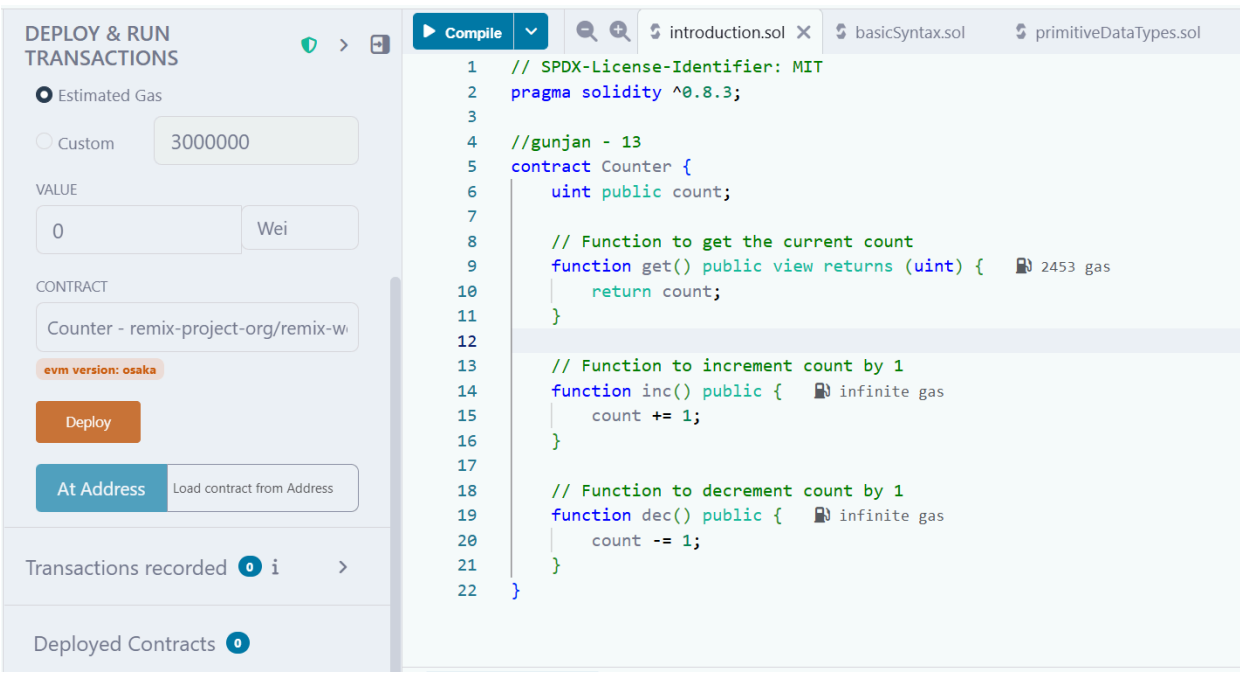
OUTPUT:

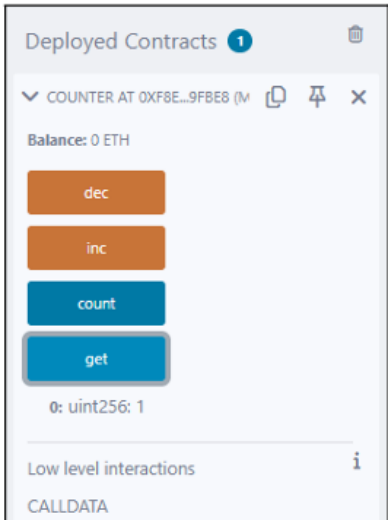
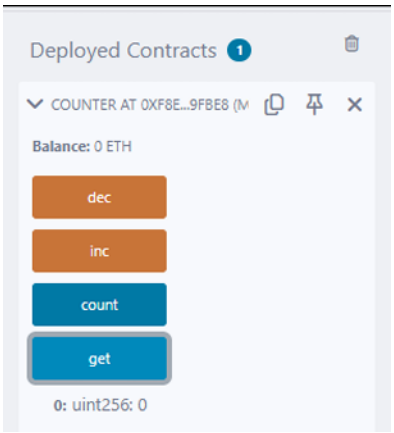
Tutorial 1:

- Tutorial no. 1 – Compile the code

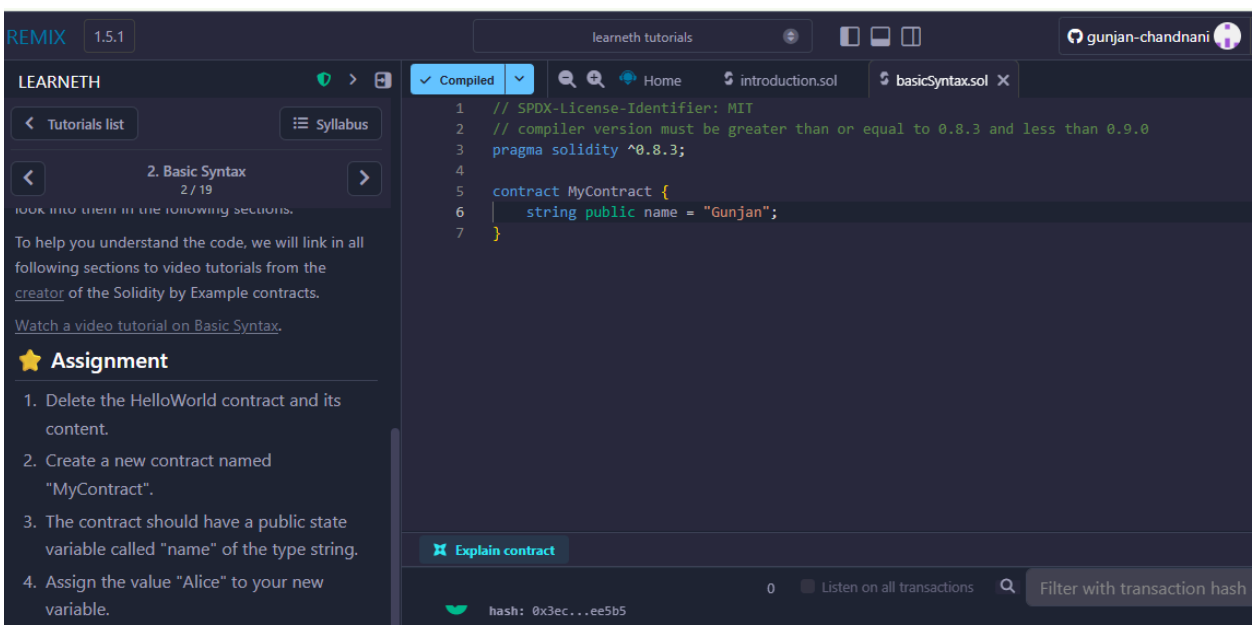


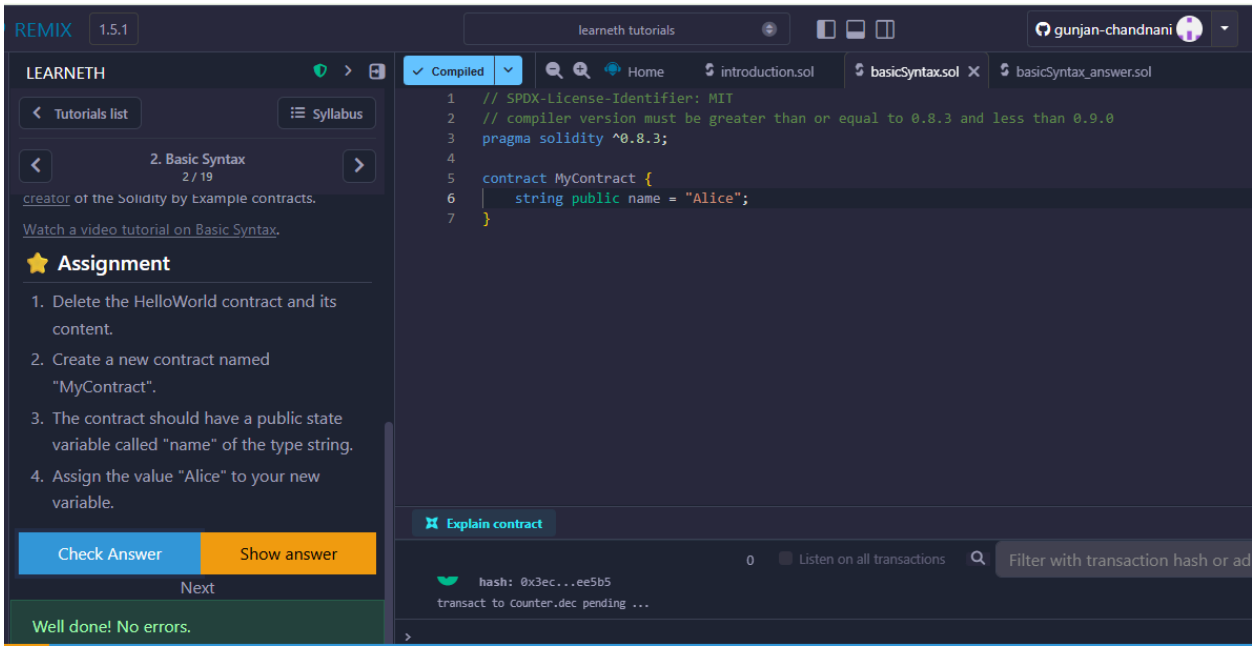
- Deploy the contract



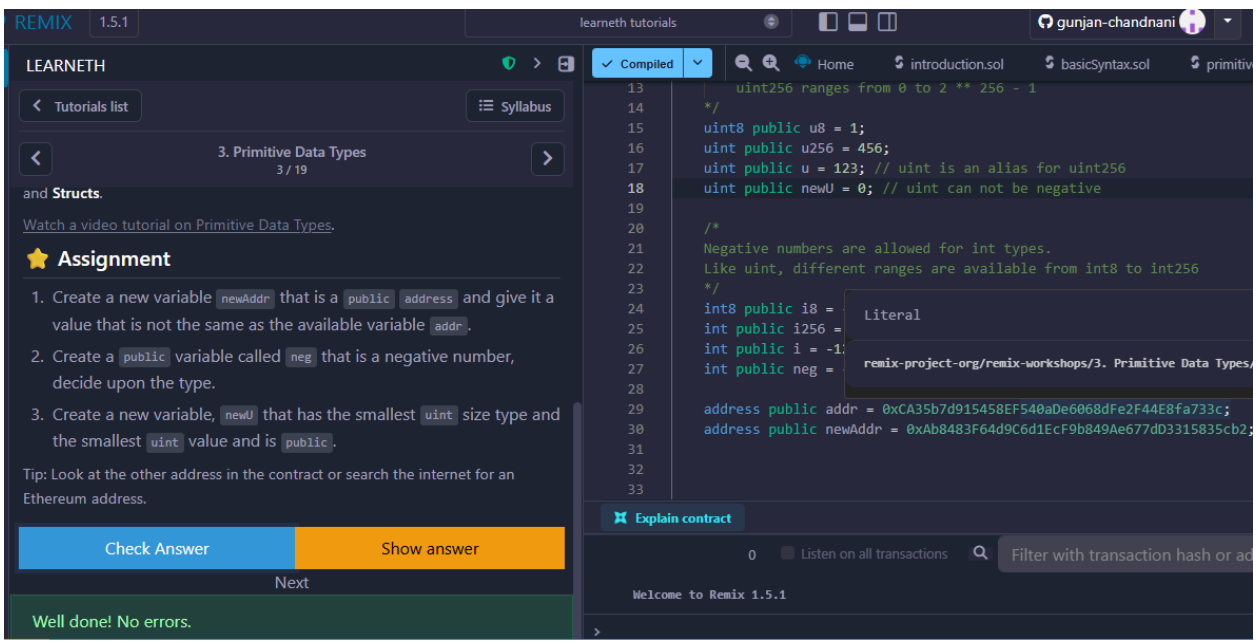
GET	Increment	Decrement
		

Tutorial 2:

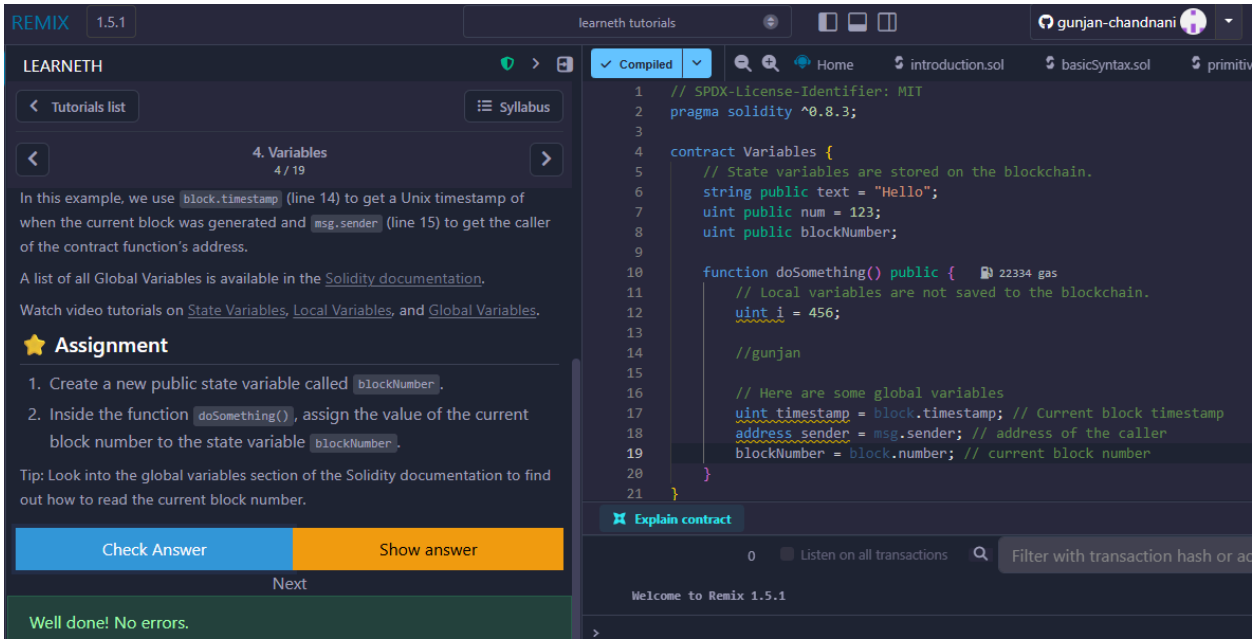



REMIX 1.5.1
learneth tutorials
gunjan-chandnani
LEARNETH
Tutorials list
Syllabus
2. Basic Syntax
2 / 19
creator of the Solidity by Example contracts.
Watch a video tutorial on Basic Syntax.
Assignment
1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.
Check Answer
Show answer
Next
Well done! No errors.
Compiled
Home
introduction.sol
basicSyntax.sol
basicSyntax_answer.sol
// SPDX-License-Identifier: MIT
// compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
pragma solidity ^0.8.3;
contract MyContract {
string public name = "Alice";
}
Explain contract
Listen on all transactions
Filter with transaction hash or ad
hash: 0x3ec...ee5b5
transact to Counter.dec pending ...

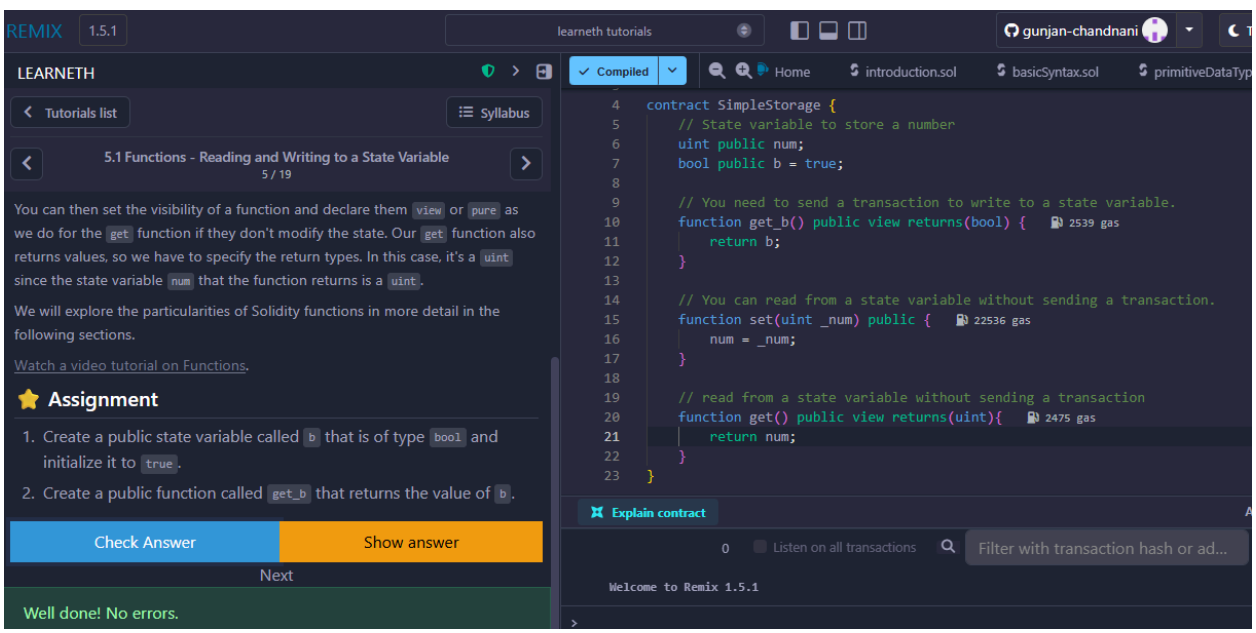
Tutorial 3:


REMIX 1.5.1
learneth tutorials
gunjan-chandnani
LEARNETH
Tutorials list
Syllabus
3. Primitive Data Types
3 / 19
and Structs.
Watch a video tutorial on Primitive Data Types.
Assignment
1. Create a new variable newAddr that is a public address and give it a value that is not the same as the available variable addr.
2. Create a public variable called neg that is a negative number, decide upon the type.
3. Create a new variable, newU that has the smallest uint size type and the smallest uint value and is public.
Tip: Look at the other address in the contract or search the internet for an Ethereum address.
Check Answer
Show answer
Next
Well done! No errors.
Compiled
Home
introduction.sol
basicSyntax.sol
uint256 ranges from 0 to 2 ** 256 - 1
*/
uint8 public u8 = 1;
uint public u256 = 456;
uint public u = 123; // uint is an alias for uint256
uint public newU = 0; // uint can not be negative
/*
Negative numbers are allowed for int types.
Like uint, different ranges are available from int8 to int256
*/
address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
address public newAddr = 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
Literal
remix-project-org/remix-workshops/3. Primitive Data Types
Explain contract
Listen on all transactions
Filter with transaction hash or ad
Welcome to Remix 1.5.1

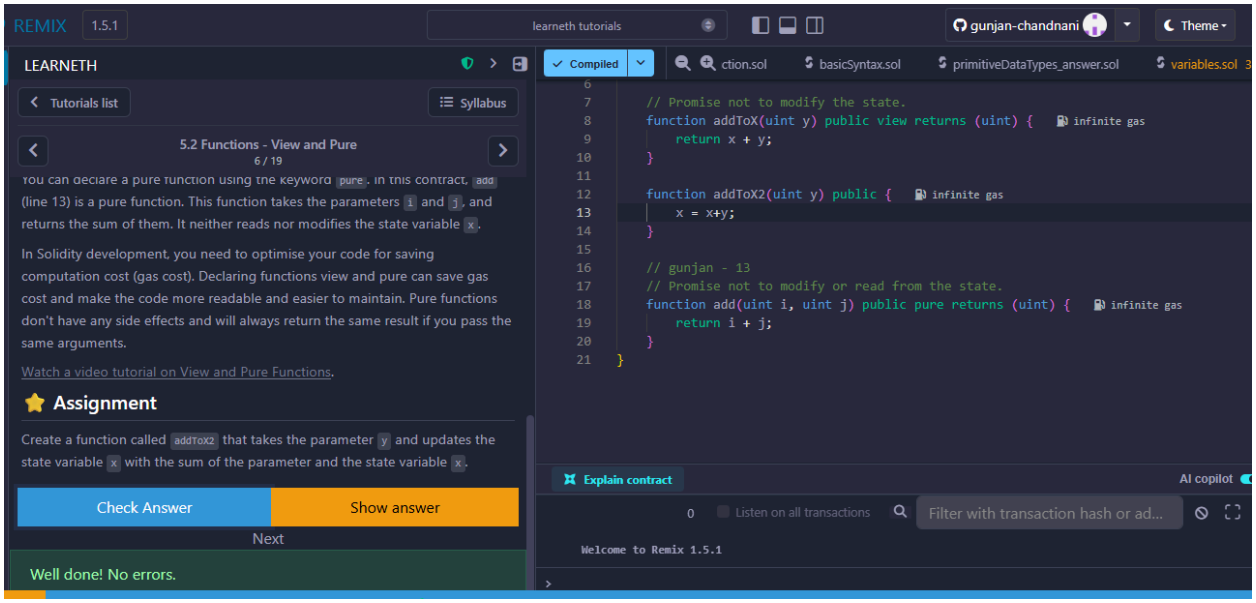
Tutorial 4:



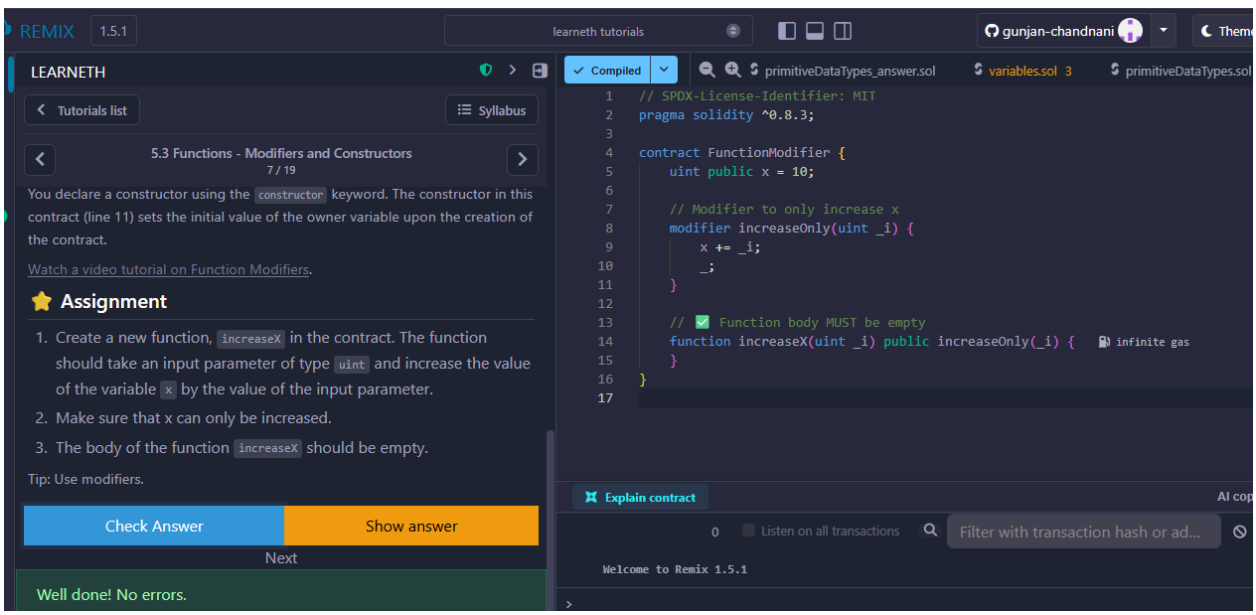
Tutorial 5:

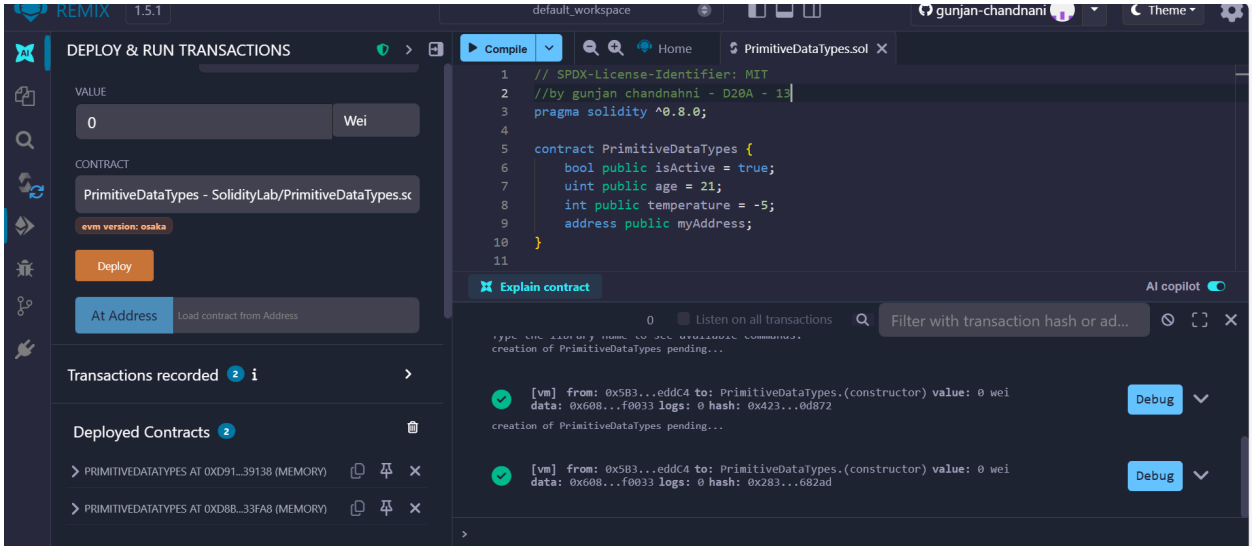


Tutorial 6:

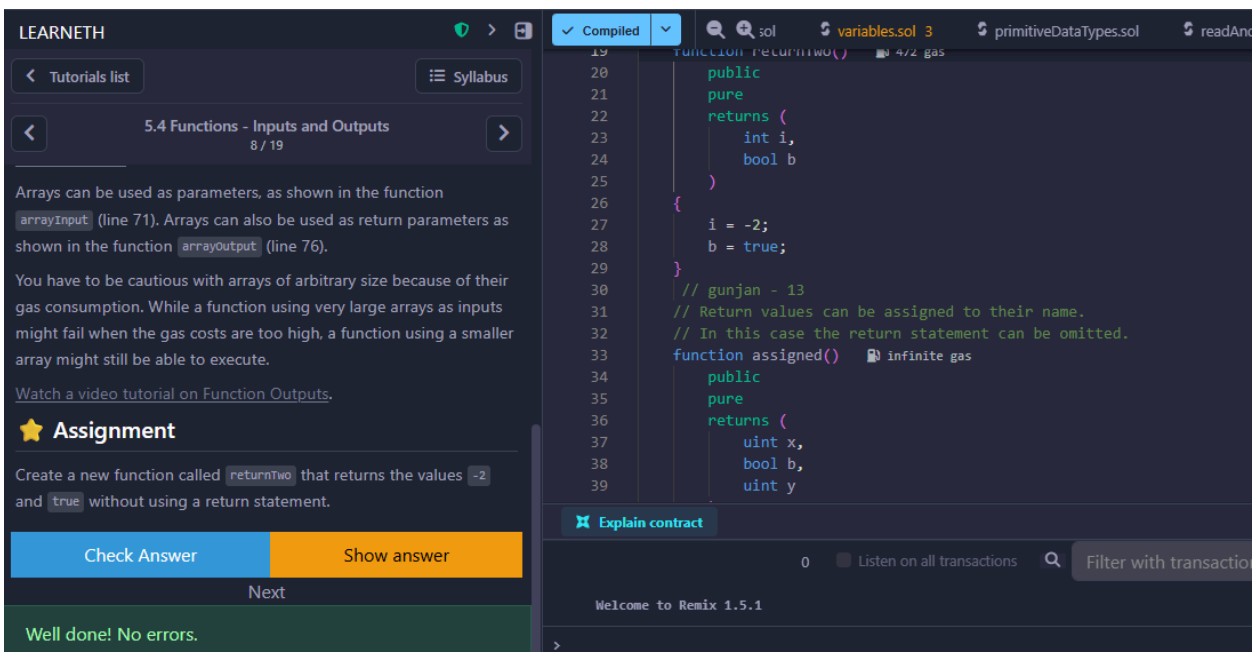


Tutorial 7:

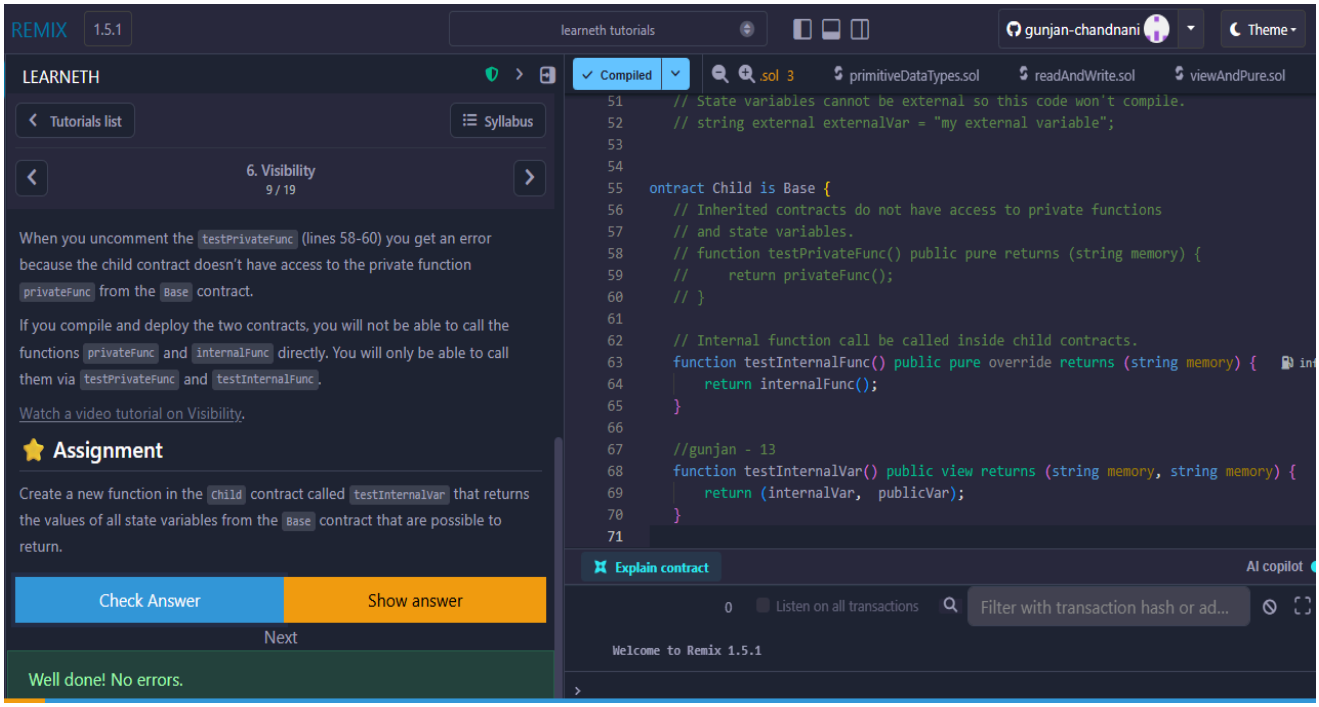



REMIX 1.5.1
default_workspace
gunjan-chandnani
Theme
DEPLOY & RUN TRANSACTIONS
VALUE
0
Wei
CONTRACT
PrimitiveDataTypes - SolidityLab/PrimitiveDataTypes.sc
evm version: osaka
Deploy
At Address
Load contract from Address
Transactions recorded 2
Deployed Contracts 2
PRIMITIVEDATATYPES AT 0XD91...39138 (MEMORY)
PRIMITIVEDATATYPES AT 0XD8B...33FA8 (MEMORY)
Compile
Home
PrimitiveDataTypes.sol
// SPDX-License-Identifier: MIT
//by gunjan chandnahni - D20A - 13
pragma solidity ^0.8.0;
contract PrimitiveDataTypes {
bool public isActive = true;
uint public age = 21;
int public temperature = -5;
address public myAddress;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
creation of PrimitiveDataTypes pending...
[vm] from: 0x5B3...eddC4 to: PrimitiveDataTypes.(constructor) value: 0 wei
data: 0x608...f0033 logs: 0 hash: 0x423...0d872
Debug
creation of PrimitiveDataTypes pending...
[vm] from: 0x5B3...eddC4 to: PrimitiveDataTypes.(constructor) value: 0 wei
data: 0x608...f0033 logs: 0 hash: 0x283...682ad
Debug

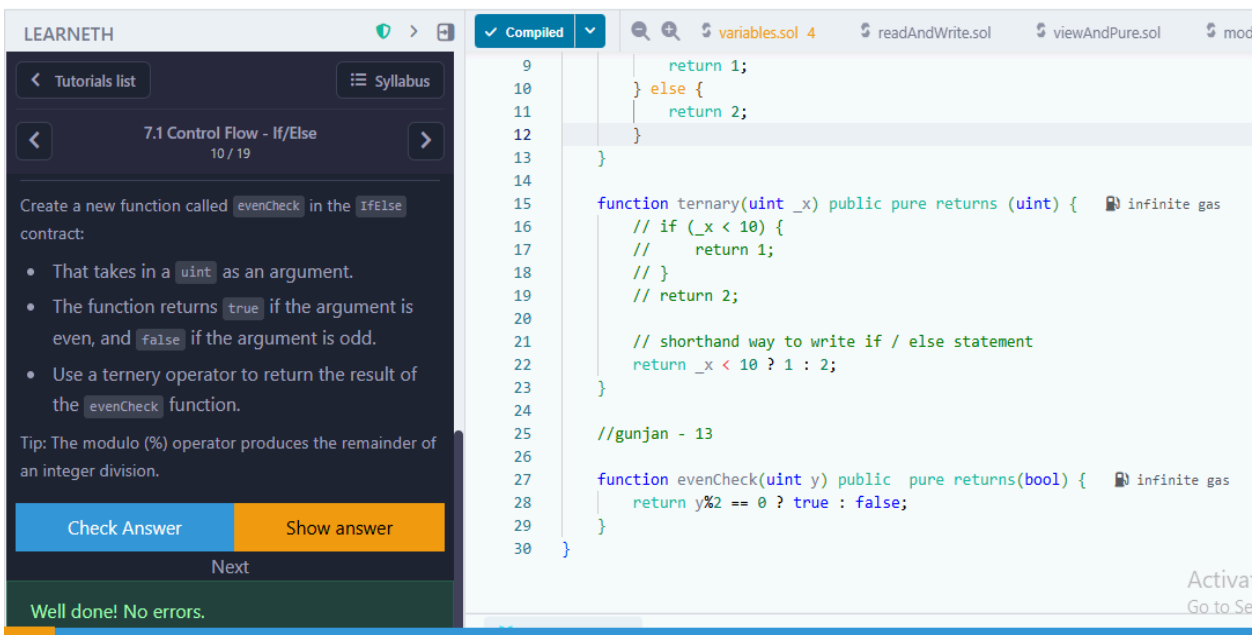
Tutorial 8:


LEARNETH
Tutorials list
Syllabus
5.4 Functions - Inputs and Outputs
8 / 19
Arrays can be used as parameters, as shown in the function arrayInput (line 71). Arrays can also be used as return parameters as shown in the function arrayOutput (line 76).
You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.
Watch a video tutorial on Function Outputs.
Assignment
Create a new function called returnTwo that returns the values -2 and true without using a return statement.
Check Answer
Show answer
Next
Well done! No errors.
Compiled
sol
variables.sol 3
primitiveDataTypes.sol
public
pure
returns (
int i,
bool b
)
{
i = -2;
b = true;
}
// gunjan - 13
// Return values can be assigned to their name.
// In this case the return statement can be omitted.
function assigned()
infinite gas
public
pure
returns (
uint x,
bool b,
uint y
Explain contract
Listen on all transactions
Filter with transactio
Welcome to Remix 1.5.1

Tutorial 9:



Tutorial 10:



Tutorial 11:

LEARNETH

Tutorials list | Syllabus

7.2 Control Flow - Loops
11 / 19

statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer | Show answer

Next

Well done! No errors.

Compiled | 4 | readAndWrite.sol | viewAndPure.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  //gunjan - 13
5  contract Loop {
6      uint public count;
7      function loop() public {    infinite gas
8          // for loop
9          for (uint i = 0; i < 10; i++) {
10             if (i == 5) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18             count++;
19         }
20
21         // while loop
22         uint j;
23         while (j < 10) {
24             j++;
```

Tutorial 12:

LEARNETH

Tutorials list | Syllabus

8.1 Data Structures - Arrays
12 / 19

choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer | Show answer

Next

Well done! No errors.

Compiled | IPure.sol | modifiersAndConstructors.sol | inputs/

```
32
33     function remove(uint index) public {    7587 gas
34         delete arr[index];
35     }
36 }
37
38 // gunjan - 13
39 contract CompactArray {
40     uint[] public arr;
41
42     function remove(uint index) public {    infinite gas
43         arr[index] = arr[arr.length - 1];
44         arr.pop();
45     }
46
47     function test() public {    infinite gas
48         arr.push(1);
49         arr.push(2);
50         arr.push(3);
51         arr.push(4);
52
53         remove(1);
54         remove(2);
55     }
```

Activate
Go to Settin

Tutorial 13:

LEARNETH

Tutorials list | Syllabus

8.2 Data Structures - Mappings
13 / 19

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer | Show answer

Next

Well done! No errors.

Compile | ifElse.sol | loops.sol | arrays.sol | arrays_answer.sol | ma

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Mapping {
5      mapping(address => uint) public balances;
6
7      function get(address _addr) public view returns (uint) {    2872 gas
8          return balances[_addr];
9      }
10
11     function set(address _addr) public {    25256 gas
12         // store the ETH balance of the address
13         balances[_addr] = _addr.balance;
14     }
15
16         //gunjan - 13
17     function remove(address _addr) public {    5566 gas
18         delete balances[_addr];
19     }
20 }
21
```

Activate Windows
Go to Settings to activate Win

Tutorial 14:

LEARNETH

Tutorials list | Syllabus

8.3 Data Structures - Structs
14 / 19

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer | Show answer

Next

Well done! No errors.

Compiled | rs.sol | inputsAndOutputs.sol | visibility.sol | ifElse.sol | loop

```
32         Todo storage todo = todos[_index];
33         return (todo.text, todo.completed);
34     }
35
36      //gunjan - 13
37     // update text
38     function update(uint _index, string memory _text) public {    infinite gas
39         Todo storage todo = todos[_index];
40         todo.text = _text;
41     }
42
43     // update completed
44     function toggleCompleted(uint _index) public {    28995 gas
45         Todo storage todo = todos[_index];
46         todo.completed = !todo.completed;
47     }
48
49     function remove(uint _index) public {    infinite gas
50         delete todos[_index];
51     }
52 }
```

Activate Windows
Go to Settings to activate

Tutorial 15:

Tutorials list | Syllabus

8.4 Data Structures - Enums
15 / 19

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer | Show answer

Next

Well done! No errors.

```
    Size public sizes;

    // Returns uint
    // Pending  - 0
    // Shipped  - 1
    // Accepted - 2
    // Rejected - 3
    // Canceled - 4
    function get() public view returns (Status) {    2605 gas
        return status;
    }

    function getSize() public view returns (Size) {    2633 gas
    return sizes;
}

    //Gunjan - 13 - D20A

    // Update status by passing uint into input
    function set(Status _status) public {    undefined gas
        status = _status;
    }

    // You can update to a specific enum like this
```

Tutorial 16:

LEARNETH

Tutorials list | Syllabus

9. Data Locations
16 / 19

f and assign it the value of myMemStruct.
Change the value of the `myMemStruct2` member `foo` to 1.

3. Create a new struct `myMemStruct3` with the data location *memory* inside the `function` `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function f return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer | Show answer

Next

Well done! No errors.

Compile | ol | enums.sol | enums_answer.sol | dataLocation

```
    function _f(    undefined gas
        uint[] storage _arr,
        mapping(uint => address) storage _map,
        MyStruct storage _myStruct
    ) internal {
        // do something with storage variables
    }

    // You can return memory variables
    function g(uint[] memory _arr) public returns (uint[] memory) {
        // do something with memory array
        arr[0] = 1;
    }

    function h(uint[] calldata _arr) external {    468 gas
        // do something with calldata array
        // _arr[0] = 1;
    }
}
```

Explain contract

0 | Listen on all transactions

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data:

Tutorial 17:

LEARNETH

Tutorials list | Syllabus

10.1 Transactions - Ether and Wei
17 / 19

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^18) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public` `uint` called `oneGWei` and set it to 1 `gwei`.
2. Create a `public` `bool` called `isOneGWei` and set it to the result of a comparison operation between 1 gwei and 10^9.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer | Show answer

Next

Well done! No errors.

Compiled | ifElse.sol | loops.sol 1 | arrays.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  //Gunjan - 13 - D20A
5
6  contract EtherUnits {
7      uint public oneWei = 1 wei;
8      // 1 wei is equal to 1
9      bool public isOneWei = 1 wei == 1;
10
11     uint public oneEther = 1 ether;
12     // 1 ether is equal to 10^18 wei
13     bool public isOneEther = 1 ether == 1e18;
14
15     uint public oneGwei = 1 gwei;
16     bool public isOneGwei = 1 gwei == 1e9;
17 }
```

Tutorial 18:

LEARNETH

Tutorials list | Syllabus

10.2 Transactions - Gas and Gas Price
18 / 19

Learn more about *gas* on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer | Show answer

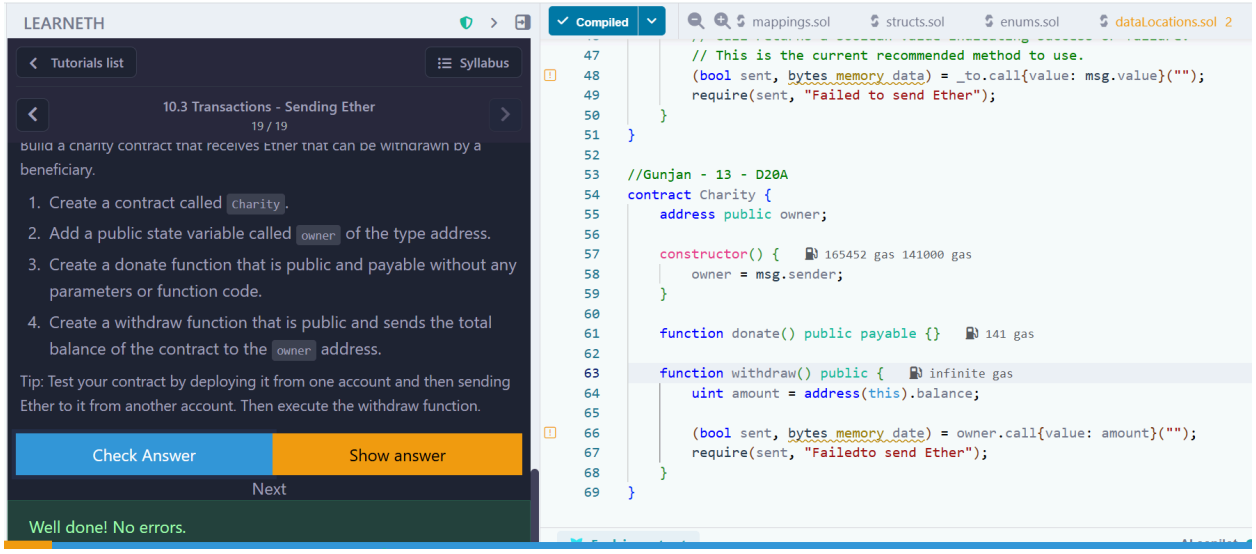
Next

Well done! No errors.

Compile | arrays.sol | mappings.sol | structs.sol | enums.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  //Gunjan - 13 - D20A
5
6  contract Gas {
7      uint public i = 0;
8      uint public cost = 170367;
9
10     // Using up all of the gas that you send causes your transaction to fail.
11     // State changes are undone.
12     // Gas spent are not refunded.
13     function forever() public {    infinite gas
14         // Here we run a loop until all of the gas are spent
15         // and the transaction fails
16         while (true) {
17             i += 1;
18         }
19     }
20 }
```

Tutorial 19:



CONCLUSION:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.